Audit de MAX Jeune Planner

Audit d'architecture, d'implémentation et de structure — 11 octobre 2026

Conclusion générale

La réflexion produit est bonne. Vous êtes exceptionnellement honnête sur ce que les données peuvent et ne peuvent pas dire aux utilisateurs, et les trois modes (planifier, explorer, comparer les dates) correspondent aux bons besoins. Mais le moteur sous-jacent ne peut pas tenir ce que l'interface promet : il renvoie silencieusement des résultats incomplets, met 20 à 70 secondes et plus par recherche, et se trompe sur les horaires autour du changement d'heure. Ce sont des problèmes d'architecture, pas des bugs à corriger un par un : il faut d'abord réparer le cœur, le reste devient ensuite plus simple.

Méthode : j'ai lu tout le code source (en ignorant `node_modules`, `out/` et l'intérieur du zip imbriqué), exécuté les tests, profilé le pipeline sur votre cache GTFS et MAX, et comparé les résultats d'Explore à un algorithme de référence simple. Votre cache ne contient qu'une seule journée (8 octobre 2026) et je n'avais pas de réseau : les chiffres sont donc indicatifs. Je n'ai pas lancé l'application empaquetée ; les points sur Electron proviennent de la lecture du code.

Ce qui est solide : la transparence sur les limites des données, l'utilisation du jeu de données SNCF des temps de correspondance minimaux, le choix de ne pas contourner DataDome, le durcissement d'Electron (sandbox, isolation de contexte, écoute locale uniquement) et une suite de tests de régression pertinente.

Critique : les résultats ne sont pas fiables

1. Le cœur de routage est une énumération exhaustive de chemins, et ce sont des plafonds qui décident des résultats

`findJourneys` et les deux fonctions d'Explore construisent tous les chemins possibles tronçon par tronçon, puis s'arrêtent à 50–60 000 expansions et tronquent les frontières à 12–15 000. Sur des données réelles, le budget est épuisé dès le deuxième tour.

Dans Explore, le budget de 50 000 est consommé pendant le traitement des chemins à 1 correspondance : le tour des chemins à 2 correspondances ou plus n'est jamais exécuté. J'ai comparé la sortie de l'application à un calcul de référence (arrivée au plus tôt) :

Origine	Réellement atteignables	Affichées par l'app	Manquantes
Lyon Part-Dieu	135	59	56 %
Rennes	128	81	37 %
Marseille St-Charles	131	54	59 %
Bordeaux St-Jean	127	76	40 %
Paris (toutes gares)	166	142	14 %

Même des destinations à 1 correspondance disparaissent : depuis Lyon, 83 sont réellement atteignables avec une correspondance, l'application en montre 19. Exemples manquants : Grenoble, Bayonne et Brest depuis Lyon ; Montpellier et Nîmes depuis Rennes.

La recherche principale souffre du même schéma. Pour Rennes → Marseille, elle atteint son budget au deuxième tour, et la frontière passe de 22 802 chemins partiels à 12 000 selon l'ordre d'insertion, ce qui favorise les départs matinaux. Elle n'atteint jamais les « jusqu'à quatre correspondances » promises par le README.

Comme la coupure est silencieuse, l'indication de l'interface « si rien n'apparaît, le jeu de données n'a pas de tels itinéraires » est fausse. De plus, dans cet instantané, l'ensemble réellement atteignable sature à 2 correspondances : les cases 3–5 correspondances ajoutent du poids à l'interface sans aucune valeur.

Correctif : remplacer l'énumération par le Connection Scan Algorithm ou RAPTOR. Trier les connexions par départ, faire un passage par tour, conserver l'arrivée au plus tôt par gare et par tour, et reconstruire les chemins par pointeurs parents. C'est exact, sans plafonds, et répond en millisecondes. Ajouter ensuite un test de propriété comparant le moteur à une référence exhaustive sur un petit jeu de données réel figé : ce test aurait détecté le problème dès le premier jour. Ma référence tenait en une quarantaine de lignes.

2. Le temps est modélisé par des objets `Date` en heure locale

Un train de nuit 22:00 → 06:00 le 24 octobre (dans la fenêtre citée par votre README, et la nuit du changement d'heure en Europe) ressort comme arrivant à 05:00 après 8 heures, au lieu de 06:00 après 9. Sur une machine dans un autre fuseau horaire, tous les instants sont décalés par rapport à l'heure de Paris, ce qui casse aussi le filtre « seulement les trains après maintenant ».

Correctif : ancrer tout sur Europe/Paris. Représenter les heures par une date de service plus des minutes depuis minuit de ce jour de service, comme GTFS, et ne convertir qu'à l'affichage.

Élevé : architecture

3. Chaque recherche refait tout le traitement des données

- `stop_times.txt` (85 Mo) est relu en flux à chaque `/api/search` (~6 s).
- `makeJourneyLegs` génère toutes les paires d'arrêts (i<j) de chaque voyage actif pour chaque jour, soit un coût en O(jours × voyages × arrêts²).
- Sur une fenêtre de 3 jours, cela représente environ 19 s avant même le routage. Sur une fenêtre de 15 jours (départ maintenant, retour dans deux semaines) : 53 000 tronçons, ~60 s rien que pour la construction, ~72 s au total, à 360–660 Mo de RSS.
- Le cache MAX ne conserve qu'une seule plage exacte `(début, fin)`. Un jour représente 11,5 k lignes, donc une fenêtre complète de 30 jours fait environ 340 k lignes (mon extrapolation). Changer d'onglet ou de date peut donc déclencher un nouveau téléchargement.

Correctif : ingérer une fois par version du flux dans un stockage indexé : voyages avec séquences d'arrêts, services développés par date, graphe des gares, et l'instantané MAX indexé par `(date, train, origine, destination)`. Le routeur doit parcourir directement les voyages au lieu de matérialiser des paires de tronçons.

Il faut aussi décider où vit ce pipeline. En local d'abord (SQLite ou index préconstruit sur chaque machine), c'est simple. Une petite tâche quotidienne qui publie un index préconstruit supprime `unzip`, rend le premier lancement instantané, et constitue la seule voie vers des alertes ou des utilisateurs hors Mac. Je penche pour cette seconde option.

4. Le moteur tourne dans le processus principal d'Electron, sans cycle de vie des requêtes

`desktop/main.js` importe `server.js` dans le processus principal : une recherche synchrone de 20 à 70 s gèlera la fenêtre. Côté client, il n'y a pas d'`AbortController`. Plusieurs réglages appellent directement `requestSubmit()` : des changements rapprochés empilent des recalculs complets, et une réponse lente plus ancienne peut écraser une plus récente.

Correctif : exécuter le moteur dans un `utilityProcess` ou un worker, donner un identifiant à chaque requête, permettre l'annulation et afficher une progression. Une fois le point 3 corrigé, c'est moins critique, mais l'isolation reste la bonne pratique.

Élevé : modèle métier

5. L'identité des gares est la couche la plus fragile

Une gare est tantôt un nom, tantôt un code IATA, tantôt un code UIC selon le chemin de code. Il existe une table de regex de regroupement (`stationKey`), une table d'alias IATA de 5 Ko en dur, une liste d'exceptions `distinctStationNames` , une coordonnée de Zürich corrigée à la main, et « Paris/Lyon toutes gares » traité en cas particulier à de nombreux endroits. Le client a son propre `stationKey` au comportement différent de celui du serveur, et les identifiants Trainline de seulement deux gares sont codés en dur dans le frontend.

Correctif : construire un registre unique des gares (identifiant, UIC, IATA, alias, coordonnées, appartenance à des groupes urbains) à partir des arrêts GTFS, des codes IATA de MAX et des référentiels SNCF. Tout le code doit utiliser des identifiants, jamais des noms, et les groupes doivent être des données.

6. Les règles d'éligibilité et de produit sont des regex dispersées, pas des données

- Bruxelles est traité en cas particulier à 5 endroits (1 serveur, 4 client). Selon la lecture que fait votre README des conditions SNCF (France, Luxembourg, Fribourg), les lignes internationales disponibles du flux incluent aussi Offenburg, Lahr et Ringsheim, qui ne reçoivent aucun avertissement. Il y a également 9 lignes OUI strictement allemandes routées comme des tronçons MAX.
- La classification OUIGO repose sur des plages de numéros de train (7xxx/4xxx) intégrées au code.
- Les `route_type 3` GTFS (autocars, 2 371 voyages) sont routés comme du « rail » et étiquetés TER.
- Le sélecteur de gares propose 3 430 noms. J'en ai compté 566 desservis uniquement par autocar et 21 uniquement par tram (par exemple « Mertzwiller Monuments aux Morts »).

Correctif : une table de règles unique avec les résultats confirmé, non confirmé ou non éligible, appliquée centralement par le moteur. Filtrer le sélecteur sur les gares ferroviaires, classées par pertinence, et étiqueter explicitement les autocars.

Produit

7. Le mode Planifier ne gère pas l'aller simple

La date de retour est `required` dans le formulaire et le serveur renvoie une erreur 400 sans elle. L'aller simple est probablement l'usage MAX le plus courant, et Explore le gère déjà. L'aller et le retour sont en outre recherchés indépendamment et ne sont appariés que dans le cas du même jour.

8. Le produit s'arrête là où se trouve la valeur

Le passage à la réservation se résume à un formulaire SNCF Connect générique, un lien profond Trainline pour deux gares, et « Copy leg details ». Trois pistes :

- **Passage à la réservation :** investir dans le meilleur lien profond vérifié par partenaire, ou indiquer explicitement qu'il n'en existe pas.
- **Surveillance et alertes :** le flux étant rafraîchi quotidiennement, « prévenez-moi quand ce tronçon passe à OUI » est la suite naturelle. Cela suppose le composant toujours actif du point 3. J'éviterais la réservation automatique : le projet `sncf-max-main` imbriqué le fait, mais il exige les identifiants de l'utilisateur et présente un risque vis-à-vis des conditions d'utilisation.
- **Chaînes de correspondances :** un trajet est composé de plusieurs tronçons réservables séparément, mais l'interface ne le dit pas. Si un tronçon se remplit ou prend du retard, rien ne protège les autres. Un état « à réserver / à vérifier » par tronçon et un indicateur clair de risque de chaîne combleraient ce manque.

9. Architecture de l'information et carte

- **Mise en page :** trois onglets de poids égal, plus les préférences de correspondance, trois bascules et cinq sections de résultats (MAX d'abord, nuit, candidats OUIGO, plus rapide, plus proche). Une seule ligne d'état mélange avertissements, fraîcheur et comptages, et une grande partie de la spécification produit vit dans un README de 14 Ko.
  - Définir un parcours principal (où puis-je aller → choisir → planifier → réserver).
  - Replier les réglages avancés.
  - Remplacer les paragraphes de mises en garde par un schéma de confiance uniforme : confirmé dans le flux / candidat OUIGO / payant / région non confirmée.
- **Carte :** la carte d'Explore est un contour SVG dessiné à la main avec une projection linéaire, sans zoom, et des marqueurs qui se chevauchent en Île-de-France et en Alsace. Pour votre fonctionnalité phare, utilisez une vraie carte (MapLibre ou équivalent) avec regroupement des marqueurs.

Structure et livraison

10. Forme du code : pas de frontières de modules

Les lignes très denses sont un symptôme. `server.js` compte en moyenne 329 caractères par ligne, avec un maximum de 2 109 (`index.html` atteint 5 434), et il regroupe HTTP, téléchargements, unzip, un parseur CSV, le cache, le filtrage GTFS, la validation et la mise en forme des réponses. `planner.js` mélange identité des gares, règles de correspondance, construction des tronçons, trois algorithmes de recherche et données d'exemple. `index.html` contient quatre blocs `<style>` ajoutés au fil du temps et du JS qui construit le HTML par concaténation de chaînes. Il y a de la duplication : deux copies des validateurs de dates, deux copies de `findReturn` , trois parseurs CSV et deux `stationKey` .

Découpage suggéré : `data/` (ingestion, index), `domain/` (gares, règles), `engine/` (CSA, explore, appariement), `api/` , `ui/` .

11. Tests et hygiène du dépôt

`npm test` échoue actuellement sur 1 test sur 18. Le test d'aller-retour dans la journée attend un retour avec environ 30 minutes sur place, alors que la durée minimale par défaut est désormais de 180. C'est exactement la dérive de règle produit que vous rencontrerez tant qu'elle vivra à plusieurs endroits. Vos trois scripts d'audit vérifient surtout la couverture des services directs, le cas facile, et dupliquent les chargeurs.

Je n'ai vu ni `.git` , ni `LICENSE` , ni configuration CI dans l'archive. Le fichier `sncf-max-main.zip` de 4 Mo à la racine du projet est un robot de réservation automatique tiers, et je n'ai trouvé aucun fichier de licence dans son contenu. Retirez-le du projet et vérifiez que rien n'en a été dérivé sans attribution.

12. Empaquetage

- Application non signée et non notarisée (pas d' `osxSign` ni d' `osxNotarize` dans la configuration Forge), arm64 uniquement, et identifiant de bundle personnel.
- Dépendance à un binaire `unzip` système.
- Le premier lancement télécharge et décompresse environ 100 Mo derrière un message statique « Loading… ».
- Pas de mise à jour automatique.

Rien de tout cela ne bloque un prototype, mais cela bloque le partage de l'application.

Ordre de travail suggéré

1. Remplacer le cœur de routage par CSA ou RAPTOR, avec le test de comparaison à la référence (points 1 et gain de vitesse du point 3).
2. Passer à un modèle de temps Europe/Paris (point 2).
3. Ajouter la couche d'ingestion et d'index, et trancher entre local et petit backend (point 3).
4. Construire le registre des gares et la table des règles d'éligibilité (points 5 et 6).
5. Isoler le moteur du processus d'interface et ajouter des requêtes annulables (point 4).
6. Travail produit : aller simple, passage à la réservation, surveillance/alertes, signalement du risque de chaîne (points 7 et 8).
7. Refonte de l'interface et vraie carte, puis empaquetage et signature (points 9 et 12).